Ex 7:

Client

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <pthread.h>

#define BUFFER_SIZE 1024

int sockfd;
int running = 1;

struct sockaddr_in serverAddr;

/* Receives messages from server */
void *receiveMessages(void *arg)
{
   char buffer[BUFFER_SIZE];
   int n;
   socklen_t addrLen;

   while (running)
   {
      addrLen = sizeof(serverAddr);

      n = recvfrom(sockfd,
               buffer,
               sizeof(buffer) - 1,
               0,
               (struct sockaddr *)&serverAddr,
               &addrLen);

      if (n < 0)
      {
         printf("Receive failed\n");
         break;
      }

      buffer[n] = '\0';

      printf("\n%s\n", buffer);
      printf("You: ");
      fflush(stdout);
   }

   return NULL;
}

int main()
{
   char buffer[BUFFER_SIZE];
```

```
    pthread_t receiveThread;

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    if (sockfd < 0)
    {
        printf("Socket creation failed\n");
        return 1;
    }

    memset(&serverAddr, 0, sizeof(serverAddr));

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(5000);
    serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

    printf("Connected to Chat Server\n");
    printf("Type 'exit' to leave the chat.\n\n");

    /* Thread for receiving messages */
    pthread_create(&receiveThread,
                   NULL,
                   receiveMessages,
                   NULL);

    while (running)
    {
        printf("You: ");

        if (fgets(buffer, sizeof(buffer), stdin) == NULL)
            break;

        buffer[strcspn(buffer, "\n")] = '\0';

        if (sendto(sockfd,
                   buffer,
                   strlen(buffer),
                   0,
                   (struct sockaddr *)&serverAddr,
                   sizeof(serverAddr)) < 0)
        {
            printf("Send failed\n");
            break;
        }

        if (strcmp(buffer, "exit") == 0)
        {
            running = 0;
            break;
        }
    }

    close(sockfd);

    return 0;
}
```

Server:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <pthread.h>
#define BUFFER_SIZE 1024
#define MAX_CLIENTS 10
struct Client { struct sockaddr_in address; };
struct Client clients[MAX_CLIENTS];
int clientCount = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
struct ClientData
{
   int sockfd;
   struct sockaddr_in clientAddr;
   char message[BUFFER_SIZE];
};
int findClient(struct sockaddr_in clientAddr)
{
   int i;
   for (i = 0; i < clientCount; i++)
      if (clients[i].address.sin_addr.s_addr == clientAddr.sin_addr.s_addr &&
         clients[i].address.sin_port == clientAddr.sin_port) return i;
   return -1;
}
void addClient(struct sockaddr_in clientAddr)
{
   if (findClient(clientAddr) == -1 && clientCount < MAX_CLIENTS)
   {
      clients[clientCount].address = clientAddr;
      clientCount++;
   }
}
void removeClient(struct sockaddr_in clientAddr)
{
   int index, i;
   index = findClient(clientAddr);
   if (index != -1)
   {
      for (i = index; i < clientCount - 1; i++) clients[i] = clients[i + 1];
      clientCount--;
   }
}
void *handleClient(void *arg)
{
   struct ClientData *data;
   int i;
   data = (struct ClientData *)arg;
   pthread_mutex_lock(&lock);
   addClient(data->clientAddr);
   printf("Client [%s:%d]: %s\n",
        inet_ntoa(data->clientAddr.sin_addr),
        ntohs(data->clientAddr.sin_port),
```

```
        data->message);
    if (strcmp(data->message, "exit") == 0)
    {
        removeClient(data->clientAddr);
        printf("Client disconnected.\n\n");
        pthread_mutex_unlock(&lock);
        free(data);
        return NULL;
    }
    for (i = 0; i < clientCount; i++)
    {
        if (clients[i].address.sin_addr.s_addr !=
                data->clientAddr.sin_addr.s_addr ||
            clients[i].address.sin_port !=
                data->clientAddr.sin_port)
        {
            char response[BUFFER_SIZE];

            sprintf(response,
                    "Client [%s:%d]: %s",
                    inet_ntoa(data->clientAddr.sin_addr),
                    ntohs(data->clientAddr.sin_port),
                    data->message);

            sendto(data->sockfd,
                   response,
                   strlen(response),
                   0,
                   (struct sockaddr *)&clients[i].address,
                   sizeof(clients[i].address));
        }
    }

    pthread_mutex_unlock(&lock);

    free(data);

    return NULL;
}

void *serverChat(void *arg)
{
    int sockfd;
    char buffer[BUFFER_SIZE];
    int i;

    sockfd = *(int *)arg;

    while (1)
    {
        printf("Server: ");

        if (fgets(buffer, sizeof(buffer), stdin) == NULL)
            break;

        buffer[strcspn(buffer, "\n")] = '\0';
```

```
        if (strcmp(buffer, "exit") == 0)
            break;

        pthread_mutex_lock(&lock);

        for (i = 0; i < clientCount; i++)
        {
            sendto(sockfd,
                   buffer,
                   strlen(buffer),
                   0,
                   (struct sockaddr *)&clients[i].address,
                   sizeof(clients[i].address));
        }

        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    int sockfd;
    char buffer[BUFFER_SIZE];

    struct sockaddr_in serverAddr, clientAddr;
    socklen_t addrLen;

    pthread_t thread;
    pthread_t serverThread;

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    if (sockfd < 0)
    {
        printf("Socket creation failed\n");
        return 1;
    }

    memset(&serverAddr, 0, sizeof(serverAddr));

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(5000);
    serverAddr.sin_addr.s_addr = INADDR_ANY;

    if (bind(sockfd,
             (struct sockaddr *)&serverAddr,
             sizeof(serverAddr)) < 0)
    {
        printf("Bind failed\n");
        close(sockfd);
        return 1;
    }

    printf("UDP Concurrent Chat Server is running...\n");
    printf("Port: 5000\n");
```

```
    printf("Waiting for clients...\n\n");

    pthread_create(&serverThread, NULL, serverChat, &sockfd);

    while (1)
    {
        int n;

        struct ClientData *data;

        addrLen = sizeof(clientAddr);

        n = recvfrom(sockfd,
                buffer,
                sizeof(buffer) - 1,
                0,
                (struct sockaddr *)&clientAddr,
                &addrLen);

        if (n < 0)
        {
            printf("Receive failed\n");
            continue;
        }

        buffer[n] = '\0';

        data = malloc(sizeof(struct ClientData));

        data->sockfd = sockfd;
        data->clientAddr = clientAddr;

        strcpy(data->message, buffer);

        pthread_create(&thread,
                NULL,
                handleClient,
                data);

        pthread_detach(thread);
    }

    close(sockfd);

    return 0;
}
```